Fondements de la programmation

Partiel 1 (2h)

Document autorisé : la feuille langage EF fournie.

Question A. Inférences de types. Parmi les expressions suivantes, lesquelles sont typables et avec quel type et lesquelles ne sont pas typables et pourquoi ?

10 pt
60 min

1. `1 + "2"` $\quad\quad$ $\mathrm{plus}(\mathrm{num}[1], \mathrm{str}[2])$
2. `if 3 >= 0 then "hi" else fun (x:num) => x` $\quad\quad$ $\mathrm{ifpos}(3, \mathrm{str}[\mathrm{hi}], \lambda\{\mathrm{num}\}(x.x))$
3. `let x = (22, 23) in Some(x) as {Some: num | None: Unit}`
4. `fun (f: str -> str) => fun (g: str -> num) => fun (x:str) => g(f(x))`
5. `(fun (x: num -> num) => x)(fun (x: num) => x)`

Question B. Filtrage de motifs. Dans les langages de programmation proposant le *pattern matching* (le filtrage de motifs) le compilateur nous prévient lorsque nous oublions un cas. Le compilateur teste que notre filtrage est bien exhaustif et nous prévient par un avertissment lorsqu'il ne l'est pas. Par exemple, soit le code OCaml suivant :

4 pt
24 min

```
type direction = Nord | Sud | Est | Ouest ;;
fun x -> match x with
  | Est -> (-1, 0)
  | Ouest -> (1, 0)
  | Sud -> (0, -1) ;;
```

Ce code définit un type (que nous noterions `{Nord: Unit | Sud: Unit | Est: Unit | Ouest: Unit}` dans EF) et applique une fonction à un argument de ce type en oubliant un cas. Le compilateur OCaml nous informe que nous avons oublié le cas `Nord`.

```
Warning 8: this pattern-matching is not exhaustive.
Here is an example of a case that is not matched:
Nord
```

En Python 3.10 (sorti le 4 octobre 2021), qui introduit une syntaxe pour le filtrage de motifs `match case` l'exhaustivité des cas n'est par contre pas testée. Selon vous qu'en est-il dans le langage EF ?

Créez une fonction qui prend en entrée un type somme et fait un filtrage de motifs en oubliant un cas (`Nord` par exemple). La fonction est-elle typable ? Conclure.

Question C. Map. Soit `T` le type `{Some: num | None: Unit}` réduire à une valeur ou une erreur l'expression suivante en justifiant chaque étape de réduction :

4 pt
24 min

```
( (fun (f: num -> num) => fun (x: T) => match x with
    | Some(x1) => Some(f(x1)) as T
    | None(x2) => None(unit) as T ) (fun (y:num) => y + 1)) (None(unit) as T)
```

Question D. Langage. Que signifie pour un langage de programmation que les fonctions y sont des citoyennes de première classe ? Donner un exemple de langage dans lesquels les fonctions ne sont pas des citoyennes de première classe et un exemple de langage dans lequel les fonctions sont des citoyennes de première classe.

2 pt
12 min

Question E. Démonstration (bonus). Prouver que si un terme e est une erreur, alors il contient soit un sous-terme $\mathrm{div}(\mathrm{num}[n], \mathrm{num}[0])$ soit un sous-terme error.